TD/TP13: Plus-courts chemins

CPES « Sciences des données, arts et cultures »

Antoine GAUQUIER
antoine.gauquier@ens.fr

Pierre ABOULKER
pierreaboulker@gmail.com

26 mai 2026

1 Exercices théoriques

1.1 Exécution de Dijkstra

Exécuter l'algorithme de Dijkstra sur le graphe présenté en Figure 1.

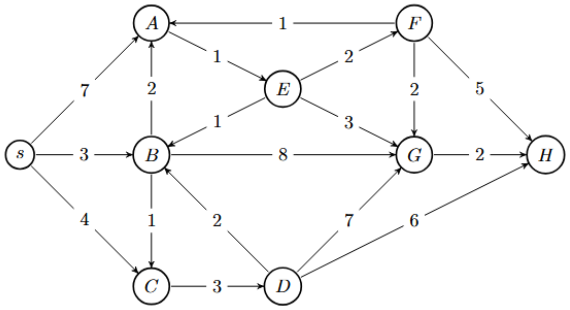


FIGURE 1 – Graphe dirigé, arc-pondéré $G = (V, E, \omega)$

1.2 Algorithme de Floyd-Warshall

Problème du *Plus courts chemins pour tout couple de sommets* :
Entrée : Un graphe dirigé, arc-pondéré $G = (V, E, \omega)$ sans cycle de poids négatif.
Sortie : Un plus court (u, v)-chemin pour chaque pair de sommet u, v.

Question 1 Résoudre le problème ci-dessus en utilisant l'algorithme de Dijkstra. Donner la complexité temporelle. Cette méthode fonctionne-t-elle sur toutes les entrées ? Justifier.

Question 2 On se propose de résoudre le problème en utilisant la programmation dynamique. La stratégie est la suivante : on fixe un ordre $(v_1, \ldots, v_n)$ des sommets de G. Pour tout $(i, j, k) \in \{1, \ldots, n\} \times \{1, \ldots, n\} \times \{0, \ldots, n\}$, on définit $SP(i, j, k)$ comme le poids d'un plus court (v_i, v_j)-chemin n'utilisant que des sommets dans $\{v_i, v_j\} \cup \{v_1, \ldots, v_k\}$. Combien de sous-problèmes sont générés par cette stratégie ?

Pour utiliser cette stratégie avec de la programmation dynamique, il faut déterminer les *cas de base* ainsi que le *cas général* (la formule récursive).

Question 3 *Cas de base* : combien vaut $SP(i, j, 0)$ quand $v_i = v_j$? Qu'en est-il quand $v_i \neq v_j$?

Question 4 *Cas général* : on considère qu'on a la solution à notre problème à l'ordre $k-1$, à savoir que l'on connait les solutions $SP(i, j, k-1)$ pour chaque $(i, j) \in \{1, \ldots, n\}^2$. Déterminer la formule de récurrence.

Question 5 Donner un pseudo-code pour cet algorithme.

Question 6 Donner les complexités temporelle et spatiale de cet algorithme.

Question 7 Dans quel(s) cas cette solution est préférable à l'utilisation de Dijkstra ? Justifier.

2 Algorithme de Dijkstra en Python

Dans cette partie, on souhaite implémenter l'algorithme de Dijkstra en Python. Lors de la séance précédente, nous avons implémenté un TAS-MIN dans lequel chaque élément est comparé uniquement via une clé numérique. Dans l'algorithme de Dijkstra, la situation est différente : chaque sommet doit être associé à une distance courante depuis la source, et le tas contient donc des *couples* de la forme `(distance, sommet)`. Ainsi, la structure de tas ne change pas, mais la notion de clé est différente : elle correspond désormais à la première composante du couple (la distance).

Nous allons compléter l'implémentation du TAS-MIN avec quelques fonctions supplémentaires, puis utiliser cette structure pour coder l'algorithme de Dijkstra. On représentera un graphe orienté et arc-pondéré par une liste d'adjacence Python. Par exemple :

```
G = {
    'A': [('B', 1), ('C', 4)],
    'B': [('C', 2), ('D', 5)],
    'C': [('D', 1)],
    'D': []
}
```

Question 1 Écrire une fonction `extraire_min(A)` retirant et retournant l'élément minimum du tas.

Question 2 Écrire une fonction `parent(i)` renvoyant l'indice du parent du sommet d'indice `i` (opération inverse de `gauche` et `droite`).

Question 3 Adapter la fonction `entasser_min(A, i)` vue en séance précédente afin qu'elle fonctionne dans le cas où les éléments du tas sont des couples `(distance, sommet)`. On supposera que la comparaison entre deux éléments du tas se fait uniquement sur la distance (première composante du couple). Faire de même pour `construction_tas_min`.

Question 4 Écrire une fonction `diminuer_cle(A, i)` restaurant la propriété de TAS-MIN après diminution de la distance associée à un sommet.

Question 5 Écrire une fonction `dijkstra(G, source)` implémentant l'algorithme de Dijkstra à l'aide du TAS-MIN précédent.

La fonction devra retourner :

— un dictionnaire `distances`;
— un dictionnaire `parents`.

Question 6 Tester votre fonction sur le graphe précédent avec `source = 'A'`. Vérifier que le résultat est correct.